

R.M.K GROUP OF ENGINEERING INSTITUTIONS



R.M.K GROUP OF INSTITUTIONS



22CS601

Compiler Design- Lab Integrated

Department of CSE

Batch/Year
2023-2027 / III Year

Created by
Dr.P.Ezhumalai, Prof. & Head / CSE-RMDEC
Mr. G. Shankar /AP - RMDEC

Mrs. D.Sterlin Rani / AP- RMDEC

Dr. G.Uma Maheswari/ASP- RMKCET

Mrs. A. Kathija Nasreen /AP -RMKCET

Mr. Thiyagarajan M /AP - RMKEC

Mrs. Bharathi B /AP -RMKEC

TABLE OF CONTENTS

S. No	Contents	Page No
1	Course Objectives	5
2	Pre Requisites	6
3	Syllabus	7
4	Course outcomes CO-	9
5	PO/PSO Mapping	10
6	Lecture Plan	11
7	Activity based learning	12
8	Lecture Notes	13
9	Assignments	48
10	Part A Questions & Answers	49
11	Part B Questions	53
12	Supportive online Certification courses	54
13	Real time Applications	55
14	Contents beyond the Syllabus	56
15	Assessment Schedule	61
16	Prescribed Text Books & Reference	62
17	Mini Project Suggestions	63
18	GATE QUESTIONS	64

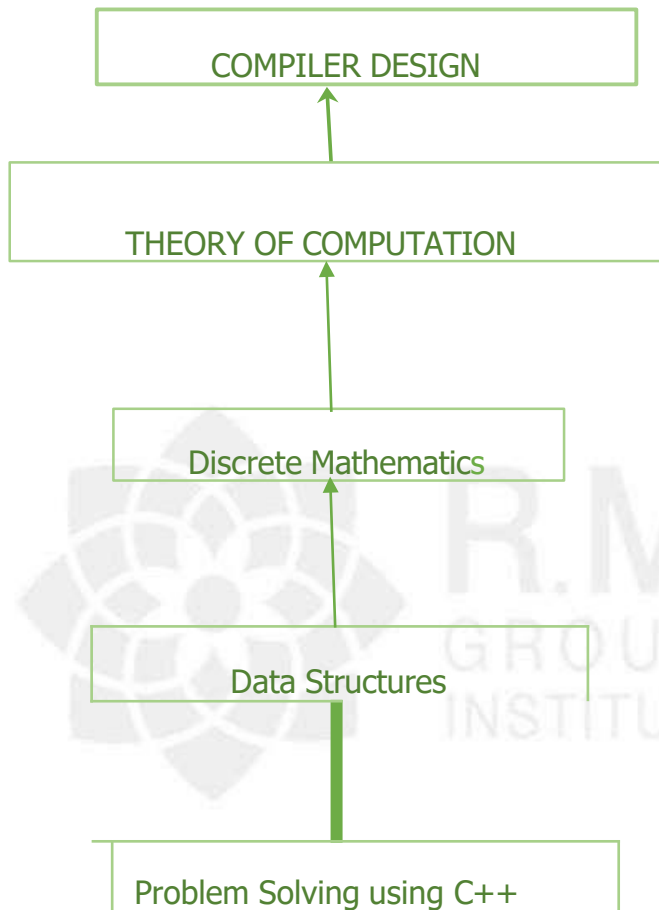
1. COURSE OBJECTIVES

- Study the different phases of compiler
- Understand the techniques for tokenization and parsing
- Understand the conversion of source program into an intermediate representation
- Learn the different techniques used for run time environment and code generation
- Analyze various code optimization techniques



2. PRE REQUI SITES

•Pre-requisite Chart



3. SYLLABUS

22CS 601 COMPI LER DESI GN

L T P

C 3 0

OBJECTIVES

2 4

- Study the different phases of compiler
- Understand the techniques for tokenization and parsing
- Understand the conversion of source program into an intermediate representation
- Learn the different techniques used for run time environment and code generation.
- Analyze various code optimization techniques

UNIT I INTRODUCTI ON TO COMPI LERS 9 + 6

Introduction–Structure of a Compiler–Role of the Lexical Analyzer - Input Buffering - Specification of Tokens - Recognition of Tokens–The Lexical Analyzer Generator LEX- Finite Automata - Regular Expressions to NFA- Optimization of DFA based pattern matches -Conversion from NFA to DFA - Minimization of DFA.

List of Exercise/Experiments:

Develop a lexical analyzer to recognize a few patterns in C. (Ex. identifiers, constants, comments, operators etc.). Create a symbol table, while recognizing identifiers.

UNIT II SYNTAX ANALYS I S 9+6

Role of the Parser - Context-free grammars – Derivation Trees – Ambiguity in Grammars and Languages- Writing a grammar – Types of parsing - Top-Down Parsing – Predictive parser or LL(1) Parser –Bottom-Up Parsing – Shift Reduce Parser - LR Parsers - SLR, CLR, LALR Parser - Parser Generators YACC.

List of Exercise/Experiments:

Design a lexical analyzer for the given language. The lexical analyzer should ignore redundant spaces, tabs and new lines, comments etc..

UNIT III INTERMEDIATE CODE GENERATI ON 9+6

Syntax Directed Definitions, Evaluation Orders for Syntax Directed Definitions, Application of Syntax Directed Translation, Intermediate Languages: Syntax Tree, Three Address Code, Implementation of Three address code – Declarations - Translation of Expressions - Type Checking.

List of Exercise/Experiments:

1. Implement a Lexical Analyzer using Lex Tool
2. Design Predictive Parser for the given language

UNIT IV RUN-TIME ENVIRONMENT AND CODE GENERATION 9 +6

Run Time Environment: Storage Organization-Storage allocation strategies - Access to nonlocal data on stack – Heap management - Parameter Passing - Issues in the design of Code Generator – Design of simple Code Generator – Register allocation and assignment.

List of Exercise/Experiments:

1. Implement an Arithmetic Calculator using LEX and YACC
2. Generate three address code for a simple program using LEX and YACC.

UNIT V CODE OPTIMIZATION 9 + 6

Principle sources of optimization – Peep hole Optimization – DAG construction - Basic blocks and flow graph - Optimization in Basic blocks – Data flow analysis.

List of Exercise/Experiments:

1. Generate three address code for a simple program using LEX and YACC.
2. Implement simple code optimization techniques (Constant folding, Strength reduction and Algebraic transformation)
3. Implement back-end of the compiler for which the three address code is given as input and the 8086 assembly language code is produced as output

TEXT BOOK:

Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, "Compilers: Principles, Techniques and Tools", Second Edition, Pearson Education Limited, 2014.

REFERENCES:

1. Randy Allen, Ken Kennedy, "Optimizing Compilers for Modern Architectures: A Dependence-based Approach", Morgan Kaufmann Publishers, 2002.
 2. Steven S. Muchnick, "Advanced Compiler Design and Implementation", Morgan Kaufmann Publishers - Elsevier Science, India, Indian Reprint, 2003.
 3. Keith D Cooper and Linda Torczon, "Engineering a Compiler", Morgan Kaufmann Publishers, Elsevier Science, 2004.
 4. V. Raghavan, "Principles of Compiler Design", Tata McGraw Hill Education Publishers, 2010.
- Allen I. Holub, "Compiler Design in C", Prentice-Hall Software Series, 1993.

4. COURSE OUTCOME

- At the end of the course, the student should be able to:

COURSE OUTCOMES		HKL
CO1	Explain the different phases of compiler	K1
CO2	Describe the fundamental components of a compiler	K1
CO3	Design and implement a lexical analyzer using finite automata and regular expressions.	K3
CO4	Compare various parsing techniques	K3
CO5	Implement code optimization techniques with simple code generators	K3
CO6	Develop code generation strategies that translate intermediate representations into target machine code.	K3

- HKL = Highest Knowledge

5. CO -PO / PSO MAPPING

CO	PROGRAM OUTCOMES												PSO		
	K3	K4	K5	K5	K3, K4, K5	A3	A2	A3	A3	A3	A3	A2	PSO 1	PS O 2	PS O 3
	PO -1	PO -2	PO -3	PO -4	PO -5	PO -6	PO -7	PO -8	PO -9	PO -10	PO -11	PO -12			
CO1	3	2	2	-	2	-	-	1	1	2	-	1	3	2	2
CO2	3	2	2	-	2	-	-	1	1	2	-	1	3	2	2
CO3	3	3	3	1	3	-	-	1	2	2	-	2	3	3	3
CO4	3	3	2	1	2	-	-	1	2	2	-	2	3	2	2
CO5	3	3	3	2	3	-	-	2	3	-	-	-	3	3	2
CO6	3	3	3	2	3	-	-	2	2	3	-	2	3	3	2

- Correlation Level -1. Slight (Low)
2. Moderate (Medium)
3. Substantial (High) , If there is no correlation, put “-”.

LECTURE PLAN

UNIT – I

S No	Topics	No of periods	Proposed date	Actual Lecture Date	pertaini ng CO	Taxonom y level	Mode of deliver y
1	Course objective, course outcome , Structure of a compiler	1	16.12.2025	16.12.2025	CO1	K1	Chalk & Talk
2	Lexical Analysis –Role of Lexical Analyzer	1	16.12.2025	16.12.2025	CO1	K1	ICT Tools
3	Input Buffering	1	19.12.2025	19.12.2025	CO1	K1	Chalk & Talk
4	Specification of Tokens , Recognition of Tokens	1	19.12.2025	19.12.2025	CO1	K2	ICT Tools
5	Lex	1	20.12.2025	20.12.2025	CO1	K2	ICT Tools
6	Finite Automata	1	20.12.2025	20.12.2025	CO1	K2	ICT Tools
7	Regular Expressions to	1	23.12.2025	23.12.2025	CO1	K3	Chalk & Talk
8	Automata	1	23.12.2025	23.12.2025	CO1	K2	Chalk & Talk
9	Minimizing DFA	1	26.12.2025	26.12.2025	CO1	K2	Chalk & Talk

7. ACTIVITY BASED LEARNING : UNI T - I

UNI T - I

1. Two-Buffer Simulation

- Create a classroom demo using two baskets as buffers.
- Students acts as “pointers” moving between buffers to simulate lexeme start and forward pointer.
- Outcome: Internal working of buffering.

2. “Compiler Assembly Puzzle

- Provide students with printed cards labeled: Lexical Analyzer, Parser, Semantic Analyzer, Intermediate Code Generator, Optimizer, Code Generator, Symbol Table, Error Handler.
- Ask groups to arrange cards in correct order and explain the flow with an example program.
- Outcome: Students visualize compiler architecture.

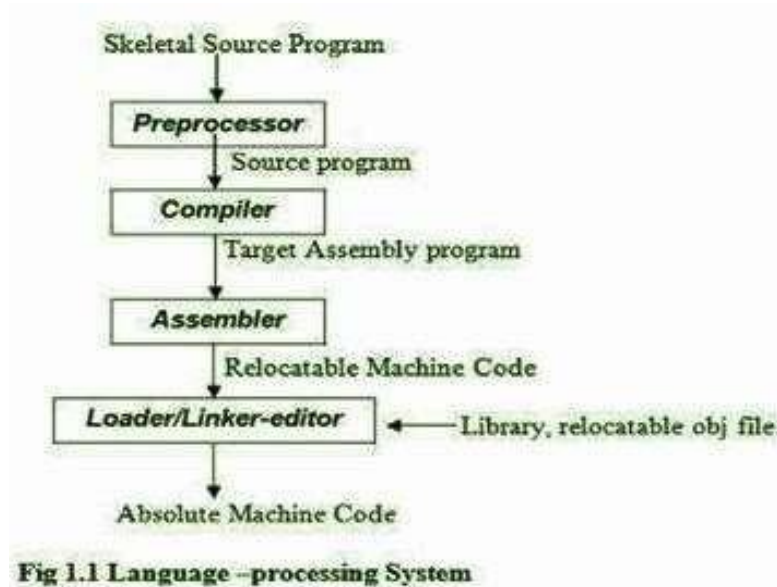
3. Create Your Own Token Rules:

- Each group designs token patterns using regular expressions: Identifier Number Operator Keyword
- Present and validate with sample inputs.
- Outcome: Understanding RE-based token specification.

8. LECTURE NOTES : UNIT - I

INTRODUCTI ON TO COMPILERS

1.1 OVERVIEW OF LANGUAGE PROCESSING SYSTEM



Preprocessor

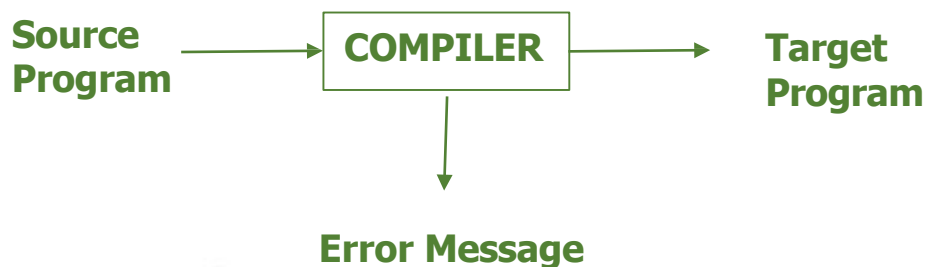
A preprocessor produce input to compilers. They may perform the following functions.

1. *Macro processing* : A preprocessor may allow a user to define macros that are short hands for longer constructs.
2. *Fileinclusion*: A preprocessor may include header files into the program text.
3. *Rational preprocessor*: these preprocessors augment older languages with more modern flow-of-control and data structuring facilities.
4. *Language Extensions*: These preprocessor attempts to add capabilities

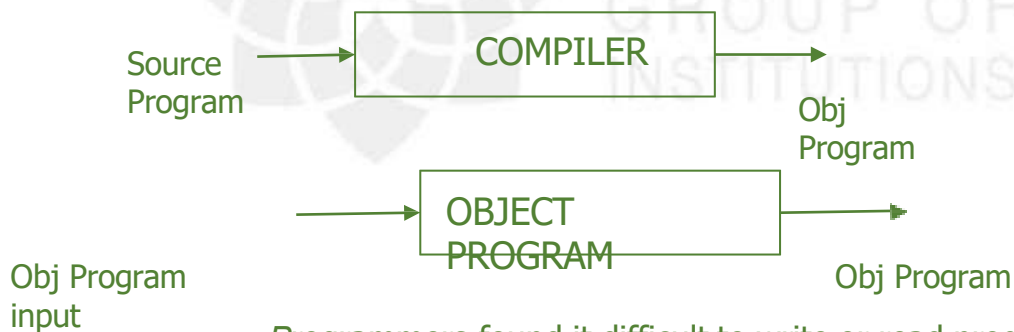
to the language by certain amounts to build-in macro

COMPILER

Compiler is a translator program that translates a program written in (HLL) the source program and translate it into an equivalent program in (MLL) the target program. As an important part of a compiler is error showing to the programmer.



Executing a program written in HLL programming language is basically of two parts. The source program must first be compiled translated into a object program. Then the results object program is loaded into a memory executed.



Programmers found it difficult to write or read programs in

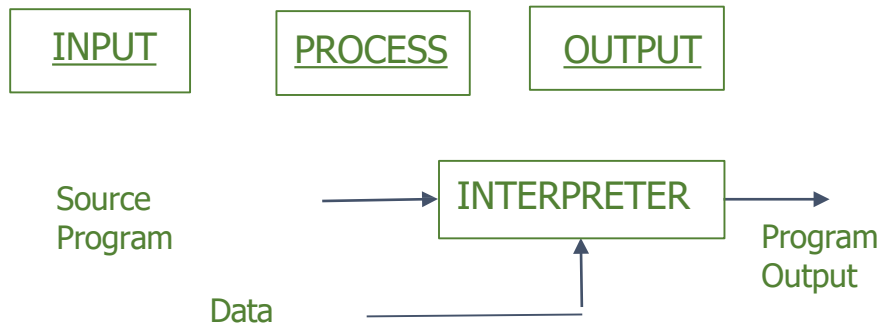
ASSEMBLER:

machine language. They begin to use a mnemonic(symbols) for each machine instruction, which they would subsequently translate into machine language.

Such a mnemonic machine language is now called an assembly language.

Programs known as assembler were written to automate the translation of assembly language into machine language. The input to an assembler program is called source program, the output is a machine language translation (objectprogram).

INTERPRETER: An interpreter is a program that appears to execute a source program as if it were machine language.



Languages such as BASIC, SNOBOL, LISP can be translated using interpreters.

JAVA also uses interpreter. The process of interpretation can be carried out in following phases.

1. Lexical analysis
2. Syntax analysis
3. Semantic analysis
4. Direct Execution

Advantages:

- Modification of user program can be easily made and implemented as execution proceeds.
- Type of object that denotes various may change dynamically.
- Debugging a program and finding errors is simplified task for a program used for interpretation.
- The interpreter for the language makes it machine independent.

Disadvantages:

- The execution of the program is *slower*. *Memory* consumption is more.

Loader and Link-editor:

Once the assembler produces an object program, that program must be placed into memory and executed. The assembler could place the object program directly in memory and transfer control to it, thereby causing the machine language program to be executed. This would waste core by leaving the assembler in memory while the user's program was being executed. Also the programmer would have to retranslate his program with each execution, thus wasting translation time. To overcome this problem of wasted translation time and memory, system programmers developed another component called loader.

"A loader is a program that places programs into memory and prepares them for execution." It would be more efficient if subroutines could be translated into object form the loader could "relocate" directly behind the user's program. The task of adjusting programs so that they may be placed in arbitrary core locations is called relocation. Relocation loaders perform four functions.

TRANSLATOR

A translator is a program that takes as input a program written in one language and produces as output a program in another language. Besides program translation, the translator performs another very important role, the error-detection. Any violation of the HLL (High Level Language) specification would be

detected and reported to the programmers. Important roles of a translator are:

- Translating the HLL program input into an equivalent ml program.
- Providing diagnostic messages wherever the programmer violates specification of the HLL.

1.8 TYPE OF TRANSLATORS:-

1) INTERPRETOR
COMPILER

2)

3) PREPROCESSOR

1.2 STRUCTURE OF THE COMPI LER DESIGN

Phases of a compiler: A compiler operates in phases. A phase is a logically interrelated operation that takes source program in one representation and produces output in another representation. The phases of a compiler are shown in below

There are two phases of compilation.

a. Analysis (Machine Independent/Language Dependent) b.Synthesis(Machine Dependent/Language independent) Compilation process is partitioned into

PHASES OF A COMPI LER

No-of-sub processes called 'phases'.

Lexical Analysis:-

LA or Scanners reads the source program one character at a time, carving the source program into a sequence of atomic units called **tokens**.

Syntax Analysis:-

The second stage of translation is called Syntax analysis or parsing. In this phase expressions, statements, declarations etc...are identified by using the results of lexical analysis. Syntax analysis is aided by using techniques based on formal grammar of the programming language.

I ntermediate Code Generations:-

An intermediate representation of the final machine language code is produced. This phase bridges the analysis and synthesis phases of translation.

Code Optimization :-

This is optional phase described to improve the intermediate code so that the output runs faster and

Code Generation:-

The last phase of translation is code generation. A number of optimizations to **reduce the length of machine language program** are carried out during this phase. The output of the code generator is the machine language program of the specified computer.

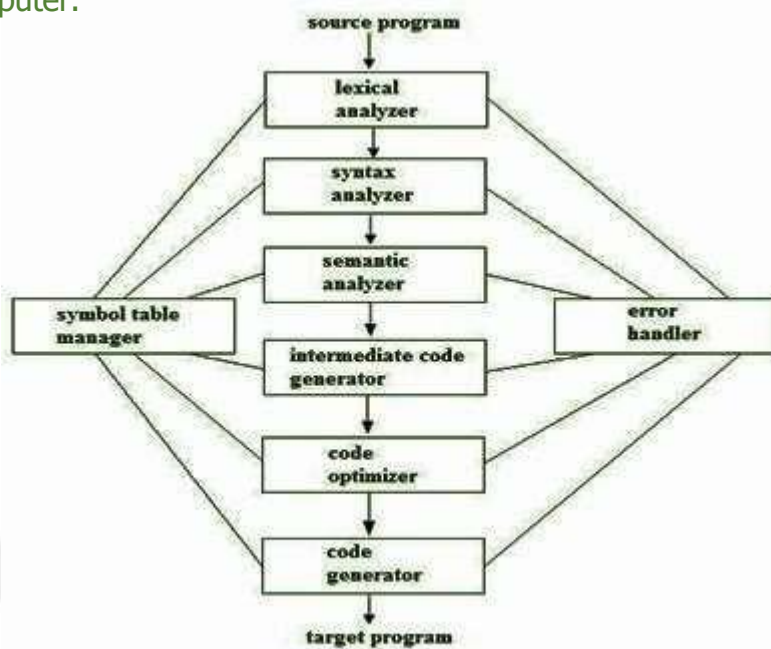


Fig 1.5 Phases of a compiler

Table Management (or) Book-keeping:-

This is the portion to **keep the names** used by the program and records essential information about each. The data structure used to record this information called a "Symbol Table".

Error Handlers:-

It is invoked when a flaw error in the source program is detected. The output of LA is

a stream of tokens ,which is passed to the next phase, the syntax analyzer or parser. The SA groups the tokens together into syntactic structure called as **expression**. Expression may further be combined to form statements. The syntactic structure can be regarded as a tree whose leaves are the token called as parse trees.

The parser has two functions . It checks if the tokens from lexical analyzer, occur in pattern that are permitted by the specification for the source language. It also imposes

on tokens a tree-like structure that is used by the sub-sequent phases of the compiler.

Example, if a program contains the expression $A+B$ after lexical analysis this expression might appear to the syntax analyzer as the token sequence `id+id`. On seeing the `/`, the syntax analyzer should detect an error situation, because the presence of these two adjacent binary operators violates the formulations rule of an expression. Syntax analysis is to make explicit the hierarchical structure of the in coming token stream by **identifying which parts of the token stream should be grouped**.

Example, ($A/B*C$ has two possible interpretations.) 1, divide A by B and then multiply by C or

2, multiply B by C and then use the result to divide A.

each of these two interpretations can be represented in terms of a parse tree.

Intermediate Code Generation:- The intermediate code generation uses the structure produced by the syntax analyzer to create a stream of simple instructions .Many styles of intermediate code are possible .One common style uses instruction with one operator and small number of operands. The output of the syntax analyzer is some representation of a parse tree. The intermediate code generation phase transforms this parse tree into an intermediate language representation of the source program.

Code Optimization This is optional phase described to improve the intermediate code so that the output runs faster and takes less space. Its output is another intermediate code program that does the some job as the original, but in a way that saves time and/or spaces.

1. Local Optimization:-

There are local transformations that can be applied to a program to make an improvement. For

L2: This can be replaced by a single statement

If $A < B$ goto L3

Another important local optimization is the elimination of common sub-

expressions

$A := B + C + D$ $E := B + C + F$

Might be evaluated as

$T1 := B + C$

$A := T1 + D$ $E := T1 + F$

Take this advantage of the common sub-expressions $B + C$.

2, Loop Optimization:- Another important source of optimization concerns about **increasing the speed of loops**. A typical loop improvement is to move a computation that produces the same result each time around the loop to a point, in the program just before the loop is entered.

Code generator:-

Cg produces the object code by deciding on the memory locations for data, selecting code to access each datum and selecting the registers in which each computation is to be done. Many computers have only a few high speed registers in which computations can be performed quickly. A good code generator would attempt to utilize registers as efficiently as possible.

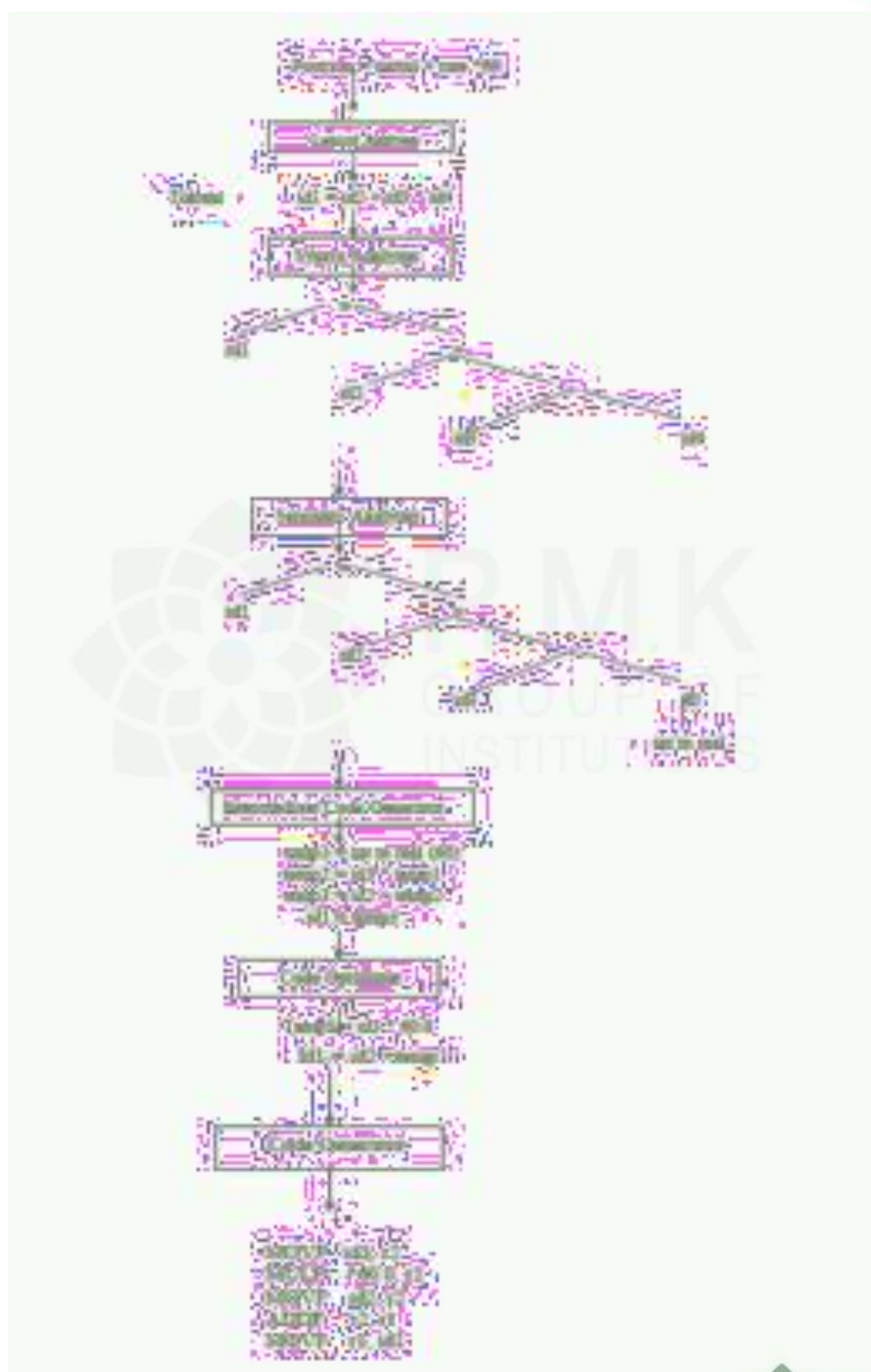
Table Management OR Book-keeping:-

A compiler needs to collect information about all the data objects that appear in the source program. The information about data objects is collected by the early phases of the compiler-lexical and syntactic analyzers. The data structure used to record this information is called as Symbol Table.

Error Handling:-

One of the most important functions of a compiler is the detection and reporting of errors in the source program. The error message should allow the programmer to determine exactly where the errors have occurred. Errors may occur in all or the phases of a compiler.

Whenever a phase of the compiler discovers an error, it must report the error to the error handler, which issues an appropriate diagnostic msg. Both of the table-management and error-handling routines interact with all phases of the compiler.



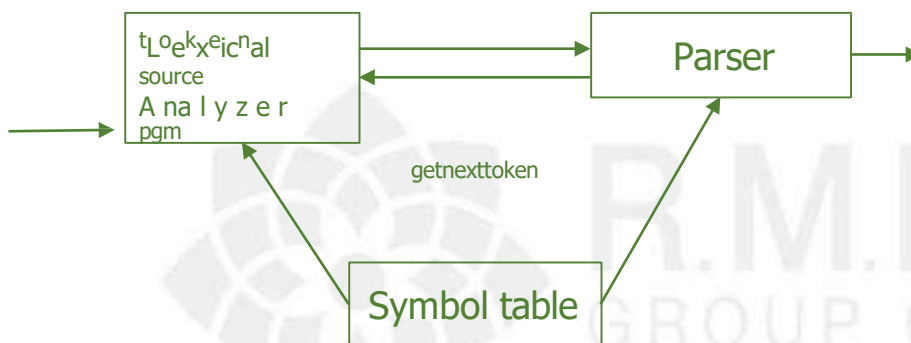
1.3 LEXICAL ANALYSIS

Lexical analysis is the process of converting a sequence of characters into a sequence of tokens. A program or function which performs lexical analysis is called a lexical analyzer or scanner. A lexer often exists as a single function which is called by a parser or another function.

THE ROLE OF THE LEXICAL ANALYZER

The lexical analyzer is the first phase of a compiler.

Its main task is used to read the input characters and produces as output a sequence of tokens that the parser uses for syntax analysis.



Upon receiving a "get next token" command from the parser, the lexical analyzer reads input characters until it can identify the next token.

ISSUES OF LEXICAL ANALYZER

There are three issues in lexical analysis:

To make the design simpler.

To improve the efficiency of the compiler. To enhance the computer portability.

TOKENS

A token is a string of characters, categorized according to the rules as a symbol (e.g., IDENTIFIER, NUMBER, COMMA). The process of forming tokens from an input stream of characters is called **tokenization**.

A token can look like anything that is useful for processing an input text stream or text file. Consider this expression in the C programming language: `sum=3+2;`

Lexeme	Token type
su	Identifier
m =	Assignment operator
3	Number
+	Addition operator
2	Number
;	End of statement

LEXEME:

Collection or group of characters forming tokens is called Lexeme.

PATTERN:

A pattern is a description of the form that the lexemes of a token may take. In the case of a keyword as a token, the pattern is just the sequence of characters that form

the keyword. For identifier and some other tokens, the pattern is a more complex structure that is matched by many strings.

Attributes for Tokens

Some tokens have attributes that can be passed back to the parser. The lexical analyzer collects information about tokens into their associated attributes. The attributes influence the translation of tokens.

a. Constant: value of the constant

b. Identifiers: pointer to the corresponding symbol table entry.

ERROR RECOVERY STRATEGIES IN LEXICAL ANALYSIS:

The following are the error-recovery actions in lexical analysis:

- 1) Deleting an extraneous character.
- 2) Inserting a missing character.
- 3) Replacing an incorrect character by a correct character.
- 4) Transforming two adjacent characters.
- 5) **Panic mode recovery:** Deletion of successive characters from the token until error is resolved.

INPUT BUFFERING

We often have to look one or more characters beyond the next lexeme before we can be sure we have the right lexeme. As characters are read from left to right, each character is stored in the buffer to form a meaningful token as shown below:

Forward pointer



Beginning of the token Look ahead pointer

We introduce a two-buffer scheme that handles large look ahead's safely. We then consider an improvement involving "sentinels" that saves time checking for the ends of buffers.

BUFFER PAIRS

A buffer is divided into two N-character halves, as shown below

Beginning of the token Look ahead pointer

We introduce a two-buffer scheme that handles large look ahead's safely. We then consider an improvement involving "sentinels" that saves time checking for the ends of buffers.

BUFFER PAIRS

A buffer is divided into two N-character halves, as shown below

Each buffer is of the same size N , and N is usually the number of characters on one disk block. E.g., 1024 or 4096 bytes.

Using one system read command we can read N characters into a buffer.

If fewer than N characters remain in the input file, then a special character, represented by **eof**, marks the end of the source file.

Two pointers to the input are maintained:

1. Pointer **lexeme beginning**, marks the beginning of the current lexeme, whose extent we are attempting to determine.
2. Pointer **forward** scans ahead until a pattern match is found.

Once the next lexeme is determined, forward is set to the character at its right end. The string of characters between the two pointers is the current lexeme. After the lexeme is recorded as an attribute value of a token returned to the parser, lexeme beginning is set to the character immediately after the lexeme just found.

Advancing forward pointer:

Advancing forward pointer requires that we first test whether we have reached the end of one of the buffers, and if so, we must reload the other buffer from the input, and move forward to the beginning of the newly loaded buffer. If the end of second buffer is reached, we must again reload the first buffer with input and the pointer

wraps to the beginning of the buffer.

Code to advance forward pointer:

```
then          begin reload  
if forward at end of first half  
second half; end
```

```
forward := forward + 1
```

```
else if forward at end of second half
```

then begin reload second half; move forward to beginning of first half end

else forward := forward + 1;

SENTINELS

For each character read, we make two tests: one for the end of the buffer, and one to

determine what character is read. We can combine the buffer-end test with the test for the current character if we extend each buffer to hold a sentinel character at the end.

The sentinel is a special character that cannot be part of the source program, and a natural choice is the character eof. The sentinel arrangement is as shown below:

Note that eof retains its use as a marker for the end of the entire input. Any eof that appears other than at the end of a buffer means that the input is at an end.

Code to advance forward pointer:

forward := forward + 1;

if forward ↑ = eof then begin

if forward at end of first half then begin reload second half;

forward := forward + 1

end

else if forward at end of second half then begin reload first half;

move forward to beginning of first half end

else / eof within a buffer signifying end of input */ terminate lexical analysis*

end

1.4 SPECIFICATION OF TOKENS

There are 3 specifications of tokens:

- 1) Strings
- 2) Language
- 3) Regular expression

Strings and Languages

An **alphabet** or character class is a finite set of symbols.

A **string** over an alphabet is a finite sequence of symbols drawn from that alphabet. A **language** is any countable set of strings over some fixed alphabet.

In language theory, the terms "sentence" and "word" are often used as synonyms for

"string." The length of a string, usually written $|s|$, is the number of occurrences of symbols in s . For example, banana is a string of length six. The empty string, denoted ϵ , is the string of length zero.

Operations on strings

The following string-related terms are commonly used:

1. A **prefix** of string s is any string obtained by removing zero or more symbols from the end of string s .

For example, ban is a prefix of $banana$.

2. A **suffix** of string s is any string obtained by removing zero or more symbols from the beginning of s . For example, $nana$ is a suffix of $banana$.

3. A **substring** of s is obtained by deleting any prefix and any suffix from s . For example, nan is a substring of $banana$.

4. The **proper prefixes, suffixes, and substrings** of a string s are those prefixes, suffixes, and substrings, respectively of s that are not ϵ or not equal to s itself.

4. A **subsequence** of s is any string formed by deleting zero or more not necessarily consecutive positions of s . For example, $baan$ is a subsequence of $banana$.

Operations on languages:

The following are the operations that can be applied to languages:

1. Union 2. Concatenation 3. Kleene closure 4. Positive closure
The following example shows the operations on strings: Let $L = \{0, 1\}$ and $S = \{a, b, c\}$

1. Union : $L \cup S = \{0, 1, a, b, c\}$
2. Concatenation : $L.S = \{0a, 1a, 0b, 1b, 0c, 1c\}$
3. Kleene closure : $L^* = \{\epsilon, 0, 1, 00, \dots\}$
4. Positive closure : $L^+ = \{0, 1, 00, \dots\}$

Regular Expressions

Each regular expression r denotes a language $L(r)$.

Here are the rules that define the regular expressions over some alphabet Σ and the

languages that those expressions denote:

1. ϵ is a regular expression, and $L(\epsilon)$ is $\{\epsilon\}$, that is, the language whose sole member is

the empty string.

1. If " a " is a symbol in Σ , then " a " is a regular expression, and $L(a) = \{a\}$, that is, the language with one string, of length one, with " a " in its one position.

2. Suppose r and s are regular expressions denoting the languages $L(r)$ and $L(s)$. Then,

a) $(r)|(s)$ is a regular expression denoting the language $L(r) \cup L(s)$.

b) $(r)(s)$ is a regular expression denoting the language

$L(r)L(s)$. c) $(r)^*$ is a regular expression denoting $(L(r))^*$.

d) (r) is a regular expression denoting $L(r)$.

4. The unary operator $*$ has highest precedence and is left associative.
5. Concatenation has second highest precedence and is left associative.

6. $|$ has lowest precedence and is left associative.

Regular set

A language that can be defined by a regular expression is called a regular set. If two regular expressions r and s denote the same regular set, we say they are

equivalent and Write $r = s$.

There are a number of algebraic laws for regular expressions that can be used to manipulate into equivalent forms.

For instance, $r|s = s|r$ is commutative; $r|(s|t) = (r|s)|t$ is associative.

Regular Definitions

Giving names to regular expressions is referred to as a Regular definition. If Σ is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form

$d_1 \rightarrow r_1$
 $d_2 \rightarrow r_2$
.....
 $d_n \rightarrow r_n$

1. Each d_i is a distinct name.
2. Each r_i is a regular expression over the alphabet $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$.

Example: Identifiers is the set of strings of letters and digits beginning with a letter.

Regular definition for this set:

$\text{letter} \rightarrow A | B | \dots | Z | a | b | \dots | z$
 $\text{digit} \rightarrow 0 | 1$

$|\dots | 9$
 $\text{id} \rightarrow \text{letter} (\text{letter} | \text{digit}) ^*$

Shorthands

Certain constructs occur so frequently in regular expressions that it is convenient to introduce notational shorthands for them.

1. One or more instances (+):

- The unary postfix operator + means "one or more instances of".
- If r is a regular expression that denotes the language $L(r)$, then $(r)^+$ is a regular expression that denotes the language $(L(r))^+$.
- Thus the regular expression a^+ denotes the set of all strings of one or more a 's.

- The operator + has the same precedence and associativity as the operator $*$.

2. Zero or one instance (?):

Zero or one instance (?):

- The unary postfix operator ? means "zero or one instance of".
- The notation $r?$ is a shorthand for $r \mid \epsilon$.
- If „ r “ is a regular expression, then $(r)?$ is a regular expression that denotes

3. the language $L(r) \cup \{\epsilon\}$.

- The notation $[abc]$ where a , b and c are alphabet symbols denotes the regular expression $a \mid b \mid c$.

- Character class such as $[a-z]$ denotes the regular expression $a \mid b \mid$

$c \mid d \mid \dots \mid z$.

- We can describe identifiers as being strings generated by the regular

expression, $[A-Za-z][A-Za-z0-9]^*$

Non-regular Set

A language which cannot be described by any regular expression is a non-regular set.

Example: These to fall strings of balanced parentheses and repeating strings cannot

be described by a regular expression. This set can be specified by a context-free grammar.

1.5 RECOGNITION OF TOKENS

Consider the following grammar fragment:

stmt $\rightarrow$ if expr then stmt | if expr then stmt else stmt | ϵ

expr $\rightarrow$ term relop term | term

term $\rightarrow$ id | num

where the terminals if, then, else, relop, id and num generate sets of strings given by

the following regular definitions:

if the n $\rightarrow$ if the n

els els $\rightarrow$ e

e $\rightarrow$

relop $\rightarrow$ < | <= | = | <> | > | >=

id $\rightarrow$ letter(letter|digit)*

num $\rightarrow$ digit+ (.digit+)?(E(+|-)
)?digit+)?

For this language fragment the lexical analyzer will recognize the keywords if, then, else,

as well as the lexemes denoted by relop, id, and num. To simplify matters, we assume keywords are reserved; that is, they cannot be used as identifiers.

Transition

diagrams It is a diagrammatic representation to depict the action that will take place when a lexical analyzer is called by the parser to get the next token. It is used to keep track of information about the characters that are seen as the forward pointer scans the input.

Transition diagram for relational operators

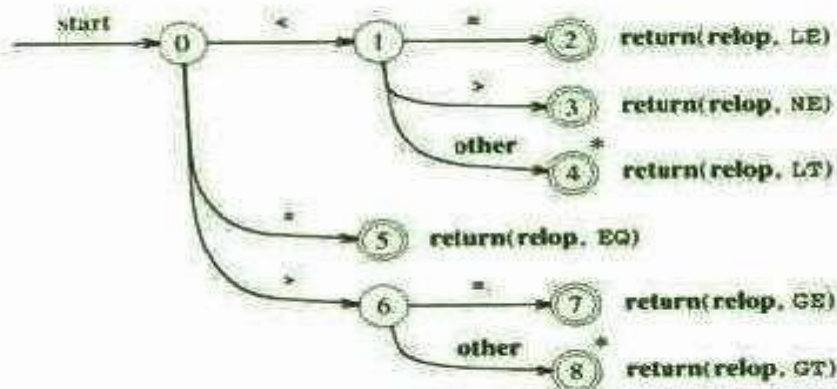
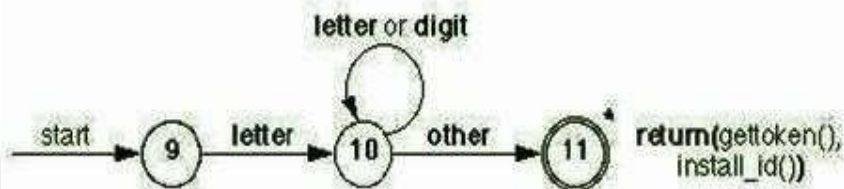


Fig. 3.12. Transition diagram for relational operators.

Transition diagram for identifiers and keywords



1.6 FINITEAUTOMATA

Finite Automata is one of the mathematical models that consist of a number of states

and edges. It is a transition diagram that recognizes a regular expression or grammar.

Types of Finite Automata

There are two types of Finite Automata:

Non-deterministic Finite Automata (NFA) Deterministic Finite Automata (DFA)

Non-deterministic Finite Automata

NFA is a mathematical model that consists of five tuples denoted by $M = \{Q_n, \Sigma, \delta, q_0, f_n\}$

Q_n –finite set of states

Σ –finite set of input symbols

δ –transition function that maps state-symbol pairs to set of states q_0 –starting state

f_n –final state

Deterministic Finite Automata

DFA is a special case of a NFA in which i) no state has an ϵ -transition.

ii) there is at most one transition from each state on any input. DFA has

five tuples denoted by

$M = \{Q, \Sigma, \delta, q_0, f\}$ Q – finite set of states

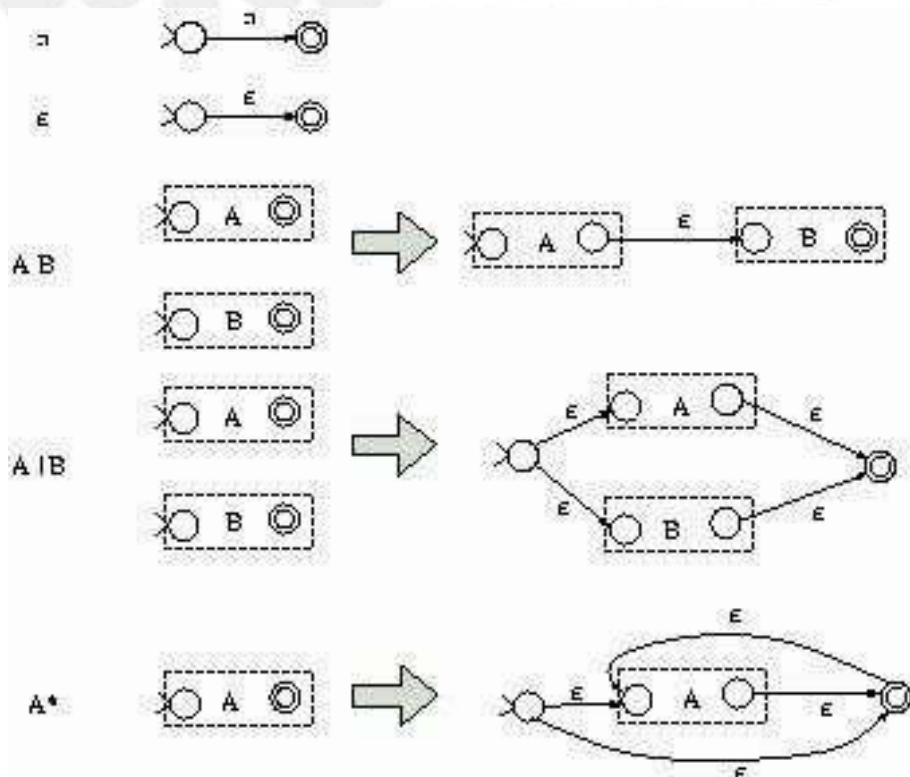
Σ – finite set of input symbols

δ – transition function that maps state-symbol pairs to set of states q_0
starting state

f – final state

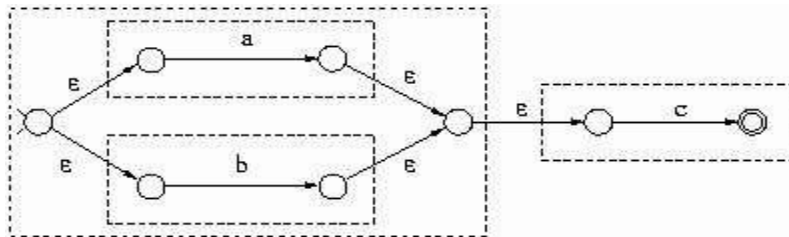
1.7 Converting a Regular Expression into a Non-Deterministic Finite Automaton (Thompson's Algorithm)

There are only 5 rules, one for each type of RE:



The algorithm constructs NFAs with only one final state. For example, the third rule indicates that, to construct the NFA for the RE AB , we construct the NFAs for A and B which are represented as two boxes with one start and one final state for each box. Then the NFA for AB is constructed by connecting the final state of A to the start state of B using an empty transition.

For example, the RE $(a|b)c$ is mapped to the following NFA:



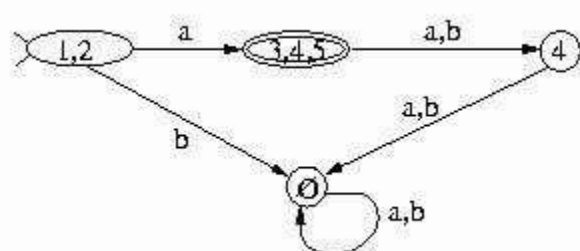
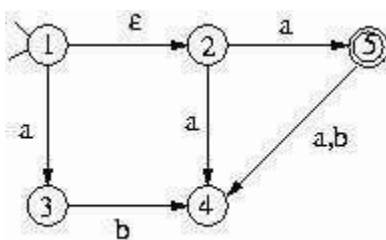
1.8 Non Deterministic Finite Automaton into a Deterministic Finite Automaton: (Subset Construction Algorithm)

Then ext step is to convert aNFA to aDFA(called *subset construction*).

Suppose that you as sign a number to each NFA state. The DFA states generated by subset construction have sets of numbers, instead of just one number. For example, aDFA

state may have been as signed the set $\{5,6,8\}$. This indicates that arriving to the state labeled $\{5,6,8\}$ in the DFA is the same as arriving to the state 5, the state 6, or the state 8 in the NFA when parsing the same input. (Re call that a particular input sequence when parsed by aDFA, leads to a unique state, while when parsed by aNFA it may lead to multiple states.)

First we need to handle transitions that lead too their states for free (without consuming any input). These are the transitions. We define the *closure* of aNFA node as the set of all the nodes reachable by this node using zero, one, or more transitions. For example, The closure of node 1 in the left figure below

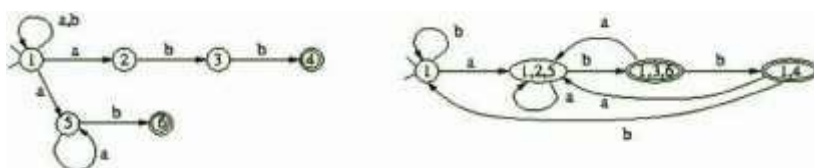


is the set $\{1,2\}$. The start state of the constructed DFA is labeled by the closure of the NFA start state. For every DFA state labeled by some set and for every character c in the language alphabet,

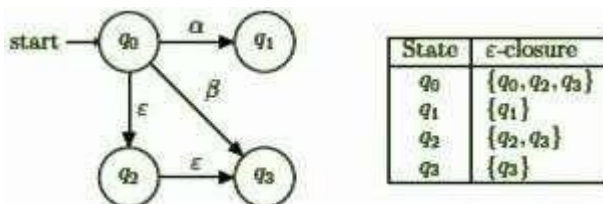
you find all the states reachable by $s_1, s_2, \dots$, or s_n using c arrows and you union together the closures of these nodes. If this set is not the label of any other node in the DFA constructed so far, you create a new DFA node with this label. For example, node $\{1,2\}$ in the DFA above has an arrow

to a $\{3,4,5\}$ for the character a since the NFA node 3 can be reached by 1 on a and nodes 4 and 5 can be reached by 2. The b arrow for node $\{1,2\}$ goes to the error node which is associated with an

empty set of NFA nodes. The following NFA recognizes , even though it wasn't constructed



Converting NFAs to DFAs To convert an NFA to a DFA, we must and away to remove all ϵ -transitions and to ensure that there is one transition per symbol in each state. We do this by constructing a DFA in which each state corresponds to a set of some states from the NFA. In the DFA, transitions from a state by some symbol go to the state S that consists of all the possible NFA-states that could be reached by from some NFA state q contained in the present DFA state S . The resulting DFA simulates the given NFA in the sense that a single DFA-transition represents many simultaneous NFA-transitions. The first concept we need is the " ϵ -closure pronounced \ epsilon closure". The ϵ -closure of an NFA state q is the set containing q along with all states in the automaton that are reachable by any number of ϵ -transitions from q . In the following automaton, the ϵ -closures are given in the table to the right:



Like wise, we can do one the ϵ -closure of a set of states to be the states reachable by ϵ -transitions from its members. In other words, this is the union of the ϵ -closure so fit elements. To convert our NFA to its DFA counterpart, we begin by taking the ϵ -closure of the start state q of our NFA and constructing a new start state S . In our DFA corresponding to that ϵ -closure. Next, for each symbol in our alphabet, we record the set of NFA states that we can reach from S_0 on that symbol. For each such set, we make a DFA state corresponding to its ϵ -closure, taking care to do this only once for each set.

Node n	nullable(n)	firstpos(n)	lastpos(n)
A leaf labeled by	true	$\emptyset$	$\emptyset$
A leaf with position i	false	{i}	{i}
$n = c1 \mid c2$	nullable(c1) or nullable(c2)	firstpos(c1) U firstpos(c2)	lastpos(c1) U lastpos(c2)
$n = c1.c2$	nullable(c1) and nullable(c2)	if (nullable(c1)) firstpos(c1) U firstpos(c2) else firstpos(c1)	if (nullable(c2)) lastpos(c1) U lastpos(c2) else lastpos(c2)
$n = c1^*$	true	firstpos(c1)	lastpos(c1)

Computation of followpos

The position of regular expression can follow another in the following ways:

- If n is a cat node with left child c1 and right child c2, then for every position i in lastpos(c1), all positions in firstpos(c2) are in followpos(i).
For cat node(.), for each position i in lastpos of its left child, the firstpos of its right child will be in follows(i).
- If n is a star node and i is a position in lastpos(n), then all positions in firstpos(n) are in followpos(i).
- For star node(*), the firstpos of that node is in followpos of all positions in lastpos of that node.

Building Syntax

tree Example

Step 1 $(a|b)^*abb \rightarrow (a|b)^*abb$

#



Node	<i>followpos</i>
1	{1, 2, 3}
2	{1, 2, 3}
3	{4}
4	{5}
5	{6}
6	-

Step 4

$A = \text{firstpos}(n0) = \{1, 2, 3\}$

$\text{Move}[A, a] = \{1, 2\}$

$= \text{followpos}(1) \cup \text{followpos}(3)$

$= \{1, 2, 3, 4\} = B$

$\text{Move}[A, b] = \{2\}$

$\text{followpos}(2) = \{1, 2, 3\} = A$

$\text{Move}[B, a] = \{1, 3\}$

$= \text{followpos}(1) \cup \text{followpos}(3) = B$

$\text{Move}[B, b] = \{2, 4\}$

$= \text{followpos}(2) \cup \text{followpos}(4)$

$= \{1, 2, 3, 5\} = C$

$\text{Move}[C, a] = \{1, 3\} = B$

$\text{Move}[C, b] = \{2, 5\}$

$= \text{Followpos}(2) \cup \text{followpos}(5) = \{1, 2, 3, 6\} = \dots$

$D \text{ Move}(D, a) = \{1, 3\} = B$

$\text{Move}(D, b) = \{2, 4\} = C$

Required DFA:

$(a|b)^*abb$
#



1.9 A LANGUAGE FOR SPECIFYING LEXICAL ANALYZER

There is a wide range of tools for constructing lexical analyzers.

- Lex
- YACC

LEX

Lex is

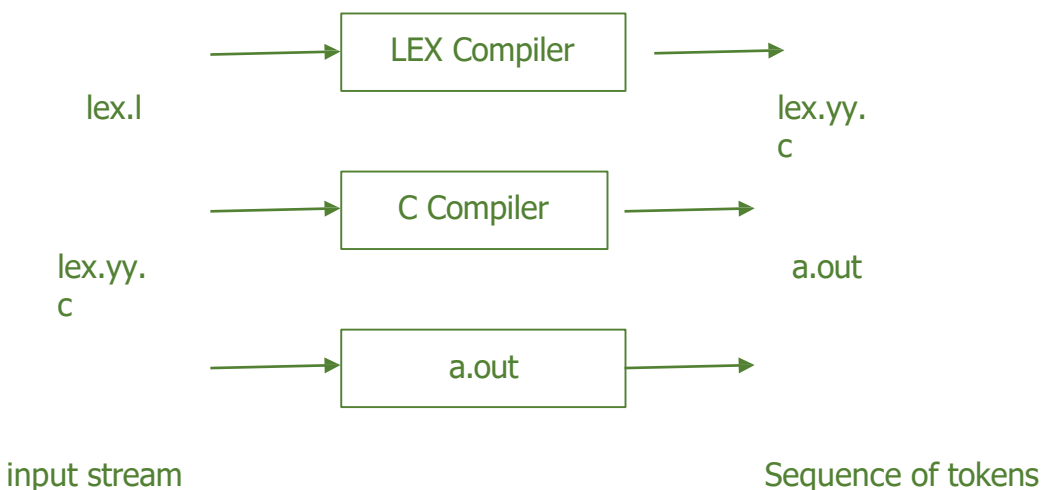
^a computer program that generates lexical analyzers. Lex is commonly used with the yacc parser generator.

Creating a lexical analyzer

First, a specification of a lexical analyzer is prepared by creating a program `lex.l` in the Lex language.

Then, `lex.l` is run through the Lex compiler to produce a C program `lex.yy.c`.

Finally, `lex.yy.c` is run through the C compiler to produce an object program `a.out`, which is the lexical analyzer that transforms an input stream into a sequence of tokens.



Lex Specification

A Lex program consists of three parts:

{definitions }

%%

{rules }

%%

{user subroutines }

Definitions include declarations of variables, constants, and regular definitions

Rules are statements of the

form $p_1 \{ \text{action } 1 \}$

$p_2 \{ \text{action } 2 \}$

...

$p_n \{ \text{action } n \}$

where p_i is regular expression and action i describes what action the lexical analyzer should take when pattern p_i matches a lexeme.

Actions are written in C code.

User subroutines are auxiliary procedures needed by the actions. These can be compiled separately and loaded with the lexical analyzer.

YACC-YET ANOTHER COMPILER-COMPILER

Yacc provides a general tool for describing the input to a computer program. The Yacc user specifies the structures of his input, together with code to be invoked as each such structure is recognized. Yacc turns such a specification into a subroutine that handles the input process; frequently, it is convenient and appropriate to have most of the flow of control in the user's application handled by this subroutine.

In the case two sets are equal, we simply reuse the existing DFA state that we already constructed. This process is then repeated for each of the new DFA states (that is, set of NFA states) until we run out of DFA states to process. Finally, every DFA state whose corresponding set of NFA states contains an accepting state is itself marked as an accepting state.

The Lexical-Analyzer Generator Lex

Lexical Analyzer tool is called Lex, or in a more recent implementation Flex, that allows one to specify a lexical analyzer by specifying regular expressions to describe patterns for tokens.

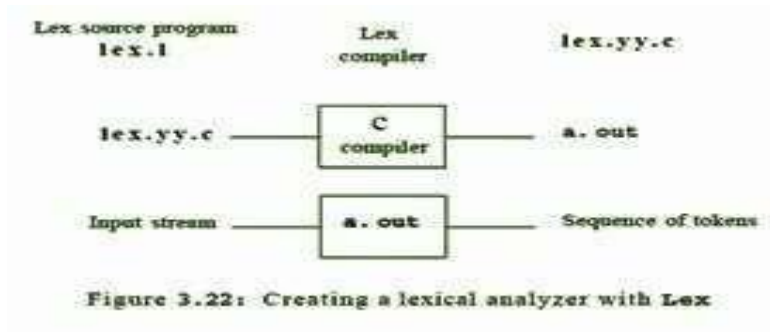
The input notation for the Lex tool is referred to as the *Lex language* and the tool itself is the *Lex compiler*.

Behind the scenes, the Lex compiler transforms the input patterns into a transition diagram and generates code, in a file called `lex.yy.c`, that simulates this transition diagram.

Use of Lex

Figure 3.22 suggests how Lex is used. An input file, which we call `lex.l`, is written in the Lex language and describes the lexical analyzer to be generated. The Lex compiler transforms `lex.l` to a C program, in a file that is always named `lex.yy.c`. The latter file is compiled by the C compiler into a file called `a.out`. The C-compiler output is a

working lexical analyzer that can take a stream of input characters and produce a stream of tokens. The normal use of the compiled C program, referred to as `asa.out` in Fig.3.22, is as a subroutine of the parser. It is a C function that returns an integer, which is a code for one of the possible token names. The attribute value, whether it be another numeric code, a pointer to the symbol table, or nothing, is placed in a global variable `yyval`, which is shared between the lexical analyzer and parser, thereby making it simple to return both the name and an attribute value of a token.



Structure of Lex Programs

A Lex program has the following form:

```

declarations
%%
translation rules
%%
auxiliary functions

```

- The declarations section includes declarations of variables, *manifest constants* (identifiers declared to stand for a constant, e.g., the name of a token), and regular definitions. The translation rules each have the form Pattern {Action }
- Each pattern is a regular expression, which may use the regular definitions the declaration section. The actions are fragments of code, typically written in C, although many variants of **Lex** using other languages have been created.
- The third section holds whatever additional functions are used in the actions.
- Alternatively, these functions can be compiled separately and loaded with the lexical analyzer.
- When called by the parser, the lexical analyzer begins reading its remaining input, one character at a time, until it finds the longest prefix of the input that matches one of the patterns P_i . It then executes the associated action A_i . Typically, A_i will return to the parser, but if it does not (e.g., because P_i describes whitespace or comments), then the lexical analyzer proceeds to find additional lexemes, until one of the corresponding actions causes a return to the parser.

The lexical analyzer returns a single value, the token name, to the parser, but uses the shared, integer variable `yylval` to pass additional information about the lexeme found, if needed.

Example: Figure 3.23 is a Lex program that recognizes the tokens of Fig. 3.12 and returns the token found.

Declarations section:

In the **declarations section** we see a pair of special brackets, `%{` and `%}`. Anything within these brackets is copied directly to the file `lex.yy.c`, and is not treated as a regular definition. It is common to place there the definitions of the manifest constants, using C `#define` statements to associate unique integer codes with each of the manifest constants.

Also in the declarations section is a **sequence of regular definitions**. Regular definitions that are used in later definitions or in the patterns of the **translation rules** are surrounded by curly braces. Thus, for instance, *delim* is defined to be a shorthand for the character class consisting of the blank, the tab, and the newline; the latter two are represented, as in all UNIX commands, by backslash followed by `t` and `n`, respectively. Then, *ws* is defined to be one or more delimiters, by the regular expression `{delim}+`.

Notice that in the definition of *id* and *number*, parentheses are used as grouping metasympols and do not stand for themselves. In contrast, *E* in the definition of *number* stands for itself. If we wish to use one of the Lex metasympols, such as any of the parentheses, `+`, `*`, or `?`, to stand for themselves, we may precede them with a backslash. For instance, we see `\.` in the definition of *number*, to represent the dot, since that character is a metasympol representing "any character," as usual in UNIX regular expressions.

Auxiliary-function section:

In the **auxiliary-function section**, we see two such functions, `installID()` and `installNum()`. Like the portion of the declaration section that appears between `%{` . . . `% }`, everything in the auxiliary section is copied directly to file `lex.yy.c`, but may be used in the actions.

Translation rules:

Finally, let us examine some of the **patterns and rules** in the middle section of Fig.3.23. First, **ws**, an identifier declared in the first section, has an associated empty action. If we find whitespace, we do not return to the parser, but look for another lexeme.

The second token has the simple regular expression pattern **if**. Should we see the two letters **if** on the input, and they are not followed by another letter or digit (which would cause the lexical analyzer to find a longer prefix of the input matching the pattern for **id**), then the lexical analyzer consumes these two letters from the input and returns the token name **IF**, that is, the integer for which the manifest

constant **IF** stands. Keywords then and else are treated similarly.

```
%{
/* definitions of manifest constants
#define LT 260
#define LE 261
EQ, NE, GT, GE, I F, THEN, ELSE, I D, NUMBER, RELOP */
}%
/* regular definitions */
[ \ t \ n]
{delim}
+ [A-Za-
delim
ws
letter
digit
id
number
%%
{ws}      {/ * no action and no return * / }
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yyval = (int) installI D(); return(I D);}
{number}  {yyval = (int) installNumO ;
return(NUMBER);}
"<"      {yyval = LT; return(RELOP) ;}
"<="     {yyval = LE; return(RELOP) ;}
"="       {yyval = EQ; return(RELOP) ;}
"<>"     {yyval = NE; return(RELOP) ;}
">"      {yyval = GT; return(RELOP) ;}
">="     {yyval = GE; return(RELOP) ;}
%%
```

```

int installID() { /* function to install the lexeme, whose first
character is pointed to by yytext, and whose length is yyleng,
into the symbol table and return a pointer thereto */

}

int installNum() { /* similar to installID, but puts numerical
constants into a separate table */

}

```

Figure 3.23: Lex program for the tokens of Fig. 3.12

The fifth token has the pattern defined by *id*. Note that, although keywords like if match this pattern as well as an earlier pattern, Lex chooses whichever pattern is listed first in situations where the longest matching prefix matches two or more patterns. The action taken when *id* is matched is threefold:

1. Function `installID( )` is called to place the lexeme found in the symbol table.
 2. This function returns a pointer to the symbol table, which is placed in global variable `yylval`, where it can be used by the parser or a later component of the compiler.
- Note that `installID( )` has available to it two variables that are set automatically by the lexical analyzer that Lex generates:

(a) `yytext` is a pointer to the beginning of the lexeme, analogous to lexeme `Begin` in Fig. 3.3.

(b) `yyleng` is the length of the lexeme found.

3. The token name `ID` is returned to the parser. The action taken when a lexeme matching the pattern *number* is similar, using the auxiliary function `installNum( )`.

Conflict Resolution in Lex

Two rules that Lex uses to decide on the proper lexeme to select, when several prefixes of the input match one or more patterns:

1. Always prefer a longer prefix to a shorter prefix.
2. If the longest possible prefix matches two or more patterns, prefer the pattern listed first in the Lex program.

Example:

- The first rule tells us to continue reading letters and digits to find the longest prefix of these characters to group as an identifier. It also tells us to treat <= as a single lexeme, rather than selecting < as one lexeme and = as the next lexeme .
- The second rule makes keywords reserved, if we list the keywords before id in the program.

For instance, if then is determined to be the longest prefix of the input that matches any pattern, and the pattern then precedes {id}, as it does in Fig.3.23, then the token THEN is returned, rather than ID.

The Lookahead Operator

• Lex automatically reads one character ahead of the last character that forms the selected lexeme, and then retracts the input so only the lexeme itself is consumed from the input.

• When we want a certain pattern to be matched to their put only when it is followed by a certain other characters. If so, we may use the slash in a pattern to indicate the end of the part of the pattern that matches the lexeme. What follows

/ is

additional pattern that must be matched before we can decide that the token in question was seen, but what matches this second pattern is not part of the lexeme.

Example: In Fortran and some other languages, keywords are not reserved.

That situation creates problems, such as a statement

I F(I ,J) = 3

where **IF** is the name of an array, not a keyword. This statement contrasts with statements of the form

I F(condition) THEN ...

where **I F** is a keyword.

To recognize the keyword **IF**, which is always followed by a left parenthesis, some text, the condition that may contain parentheses, a right parenthesis and a letter. Then, we could write a **Lex** rule for the keyword **IF** like:

I F / \ (. * \) {letter}

This rule says that the pattern the lexeme matches is just the two letters **IF**. The slash says that additional pattern follows but does not match the lexeme.

In this pattern, the first character is the left parentheses. Since that character is a **Lex** meta symbol, it must be preceded by a backslash to indicate that it has its literal meaning. The dot and star match "any string without a newline." Note that the dot is a **Lex** meta symbol meaning "any character except newline." It is followed by a right parenthesis, again with a backslash to give that character its literal meaning.

The additional pattern is followed by the symbol *letter*, which is a regular definition representing the character class of all letters.

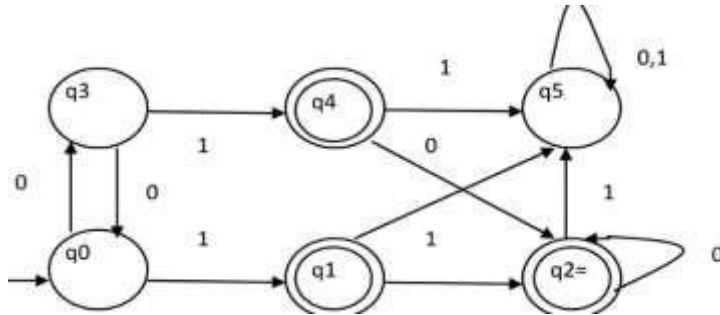
For instance, suppose this pattern is asked to match a prefix of input:

IF(A<(B+C)*D)THEN...

the first two characters match **IF**, the next character matches **\ (**, the next nine characters match **.***, and the next two match **\)** and *letter*. Note the fact that the first right parenthesis (after C) is not followed by a letter is irrelevant; we only need to find some way of matching the input to the pattern. We conclude that the letters **IF** constitute the lexeme, and they are an instance of token **if**.

9. ASSIGNMENT : UNI T - I (CO2, K3)

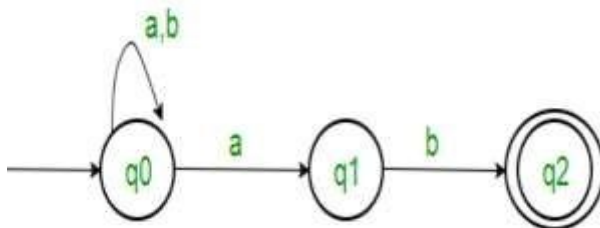
1. Minimization of DFA.



2. Minimize DFA using Thompson Construction

$((c|a)b^*)^*$

3. Conversion of NFA to DFA



4. Construct a DFA for the language over $\{0,1\}$ where the number of 1's is divisible by 3. Draw the state diagram and transition table

Convert the given NFA to an equivalent DFA:

States: $\{q_0, q_1, q_2\}$, Alphabet: $\{a,b\}$,

Transitions: $q_0 \xrightarrow{a} q_0, q_1$; $q_0 \xrightarrow{b} q_0$; $q_1 \xrightarrow{b} q_2$; $q_2 \xrightarrow{a,b} q_2$

Start state: q_0 , Final state: q_2

10. PART A : Q & A : UNI T - I

SNo	Questions and Answers	CO	K
1	What are the two parts of a compilation? Explain briefly. Analysis and Synthesis are the two parts of compilation. (Front-end & Back-end) • The analysis part breaks up the source program into constituent pieces and creates an intermediate representation of the source program. • The synthesis part constructs the desired target program from the intermediate representation.	CO1	K1
2	List the various compiler construction tools. • Parser generators • Scanner Generator • Syntax-directed translation engines • Automatic code generators • Dataflow Engines • Compiler construction tool kits		K2
3	Differentiate compiler and interpreter. • The machine-language target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs. • An interpreter, however, can usually give better error diagnostics than a compiler, because it executes the source program statement by statement.		K1
4	Define tokens, Patterns, and lexemes. • Tokens-Sequence of characters that have a collective meaning. A token of a language is a category of its lexemes. • Patterns-There is a set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a pattern associated with the token. • Lexeme-A sequence of characters in the source program that is matched by the pattern for a token.		K2
5	Describe the possible error recovery actions in lexical analyzer. •Panic mode recovery •Deleting an extraneous character •Inserting a missing character •Replacing an incorrect character by a correct character •Transposing two adjacent characters		

10. PART A : Q & A : UNI T

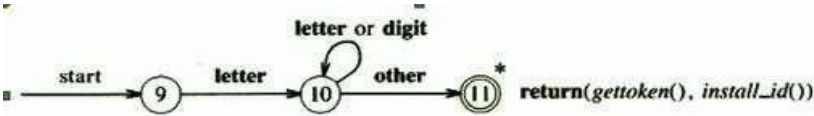
- I

SNo	Questions and Answers	CO	K
6	Write the regular expression for identifier. letter -> A B Z a b z digit -> 0 1 9 id -> letter (letter digit) *	CO1	K2
7	Write the regular expression for Number. digit -> 0 1 9 digits -> digit digit* optional_fraction->.digits € optional_exponent -> (E(+ - €)digits) € num -> digits optional_fractionoptional_exponent id -> letter (letter digit) *		K2
8	List the phases that constitute the front end of a compiler. The front end consists of those phases or parts of phases that depend primarily on the source language and are largely independent of the target machine. These include • Lexical and Syntactical analysis • The creation of symbol table • Semantic analysis • Generation of intermediate code A certain amount of code optimization can be done by the front end as well and includes Error handling that goes along with each of these phases.		K2
9	Mention the back-end phases of a compiler. The back end of compiler includes those portions that depend on the target machine and generally those portions do not depend on the source language, just the intermediate language. These include Code optimization Code generation, along with error handling and symbol-table operations.		K2
10	What is the role of lexical analysis phase? Lexical analyzer reads the source program one character at a time, and grouped into a sequence of atomic units called tokens. Identifiers, keywords, constants, operators and punctuation symbols such as commas, parenthesis, are typical tokens. Identifiers are entered into the Symbol Table. Removes comments, whitespaces (blanks, newline, tab) Keeps track of the line numbers.		K2

10. PART A : Q & A : UNI T - I

SNo	Questions and Answers	CO	K
11	What is Sentinel ? For each character read, we make two tests in lexical analysis phase: one for the end of the buffer, and one to determine what character is read(the latter may be a multi way branch).We can combine the buffer-end test with the test for the current character if we extend each buffer to hold a sentinel character at the end. The sentinel is a special character that cannot be part of the source program, and a natural choice is the character eof.	CO1	K1
12	Define Symbol Table. A symbol table is a data structure containing a record for each identifier, with fields for the attributes of the identifier. The data structure allows us to find the record for each identifier quickly and to store or retrieve data from that record quickly. Whenever an identifier is detected by a lexical analyzer, it is entered into the symbol table. The attributes of an identifier cannot be determined by the lexical analyzer.		K2
13	Define a preprocessor. A source program may be divided into modules stored in separate files. The task of collecting the source program is sometimes entrusted to a separate program, called a preprocessor.The preprocessor may also expand shorthands, called macros, into source language statements.		K2
14	What are the functions of the pre-processors? The task of collecting the source program is sometimes entrusted to a separate program, called a preprocessor. The preprocessor may also expand shorthands, called macros, into source language statements.		K2
15	What are the various parts in LEX program? Lex program will be in following form declarations %% translation rules %% auxiliary functions		

10. PART A : Q & A : UNI T - I

SNo	Questions and Answers	CO	K
16	How will you group the phases of the compiler? The front-end phases of lexical analysis, syntax analysis, semantic analysis, and intermediate code generation might be grouped together into one pass. Code optimization might be an optional pass. Then there could be a back-end pass consisting of code generation for a particular target machine.	CO1	K1
17	Name some variety of intermediate forms. · Postfix notation or polish notation. · Syntax tree · Three address code · Quadruple · Triple		K1
18	Write a regular expression to describe a languages consists of strings made of even numbers a and b. $r = ((a b)(a b))^*$		K1
19	 Fig. 3.13. Transition diagram for identifiers and keywords.		K1
20	Write a short notes on LEX. A LEX source program is a specification of lexical analyzer consisting of set of regular expressions together with an action for each regular expression. The action is a piece of code, which is to be executed whenever a token specified by the corresponding regular expression is recognized. The output of a LEX is a lexical analyzer program constructed from the LEX source specification.		

11. PART B QUESTIONS : UNIT - I

(CO1, K2)

1. Describe the various phases of compiler and trace it with the program segment (position := initial + rate * 60).
2. Explain in detail the process of compilation. Illustrate the output of each phase of the compilation for the input "a = (b+c) * (b+c) * 2"
3. Discuss input buffering techniques in detail.
4. Construct Regular expression to NFA for the sentence (alb)*a.
5. Construct NFA using the regular expression (a/b)*abb.
6. Construct the NFA from the (a|b)*a(a|b) using Thompson's construction algorithm.
7. Construct the DFA for the augmented regular expression (a | b)* # directly using syntax tree.
8. Write an algorithm for minimizing the number of states of a DFA.

PART C QUESTIONS (CO1, K3)

1. Explain the structure of a LEX program with an example.
2. Illustrate how does LEX work?
3. Consider the regular expression below which can be used as part of a specification of the definition of exponents in floating-point numbers. Assume that the alphabet consists of numeric digits ('0' through '9') and alphanumeric characters ('a' through 'z' and 'A' through 'Z') with the addition of a selected small set of punctuation and special characters. (say in this example only the characters '+' and '-' are relevant). Also, in this representation of regular expressions the character '.' Denotes concatenation.

Exponent = (+|-|€) . (E | e) . (digit)+

(i) Derive an NFA capable of recognizing its language using Thompsons' construction.

(i) Derive the DFA for the NFA found in i) above using subset construction. (iii) Minimize the DFA found in (ii) above.

1. Write Lex program to identify the following tokens -relational operators , arithmetic operators and keywords (if, while, do, switch, for).

12. Supportive online Certification courses

NPTEL : <https://nptel.ac.in/courses/106/105/106105190/>

Swayam : <https://www.classcentral.com/course/swayam-compiler-design-12926>

coursera : <https://www.coursera.org/learn/nand2tetris2>

Udemy : <https://www.udemy.com/course/introduction-to-compiler-construction-and-design/>

Mooc: <https://www.mooc-list.com/course/compiler-coursera>

edx: <https://www.edx.org/course/compiler>



13. Real time Applications in day to day life and to Industry

1. Structure of a Compiler

Used in developing programming languages and executing programs efficiently on real systems.

2. Role of the Lexical Analyzer

Used in code editors for syntax highlighting and automatic tokenization of source code.

3. Input Buffering

Used in high-speed file reading such as compilers and streaming applications to reduce I/O time.

4. Specification of Tokens

Used in search engines and NLP tools to identify meaningful words from text.

5. Recognition of Tokens

Used in log analyzers and data parsers to detect keywords and operators in large datasets.

6. Lexical Analyzer Generator (LEX)

Used to build custom scanners for domain-specific languages and configuration file parsers.

7. Finite Automata

Used in digital circuits and control systems like elevators or vending machines.

8. Regular Expressions to NFA

Used in regex engines to convert patterns into state machines for fast text matching.

9. Conversion from NFA to DFA

Used in virus scanners and intrusion detection systems for high-speed pattern scanning.

10. Minimization of DFA

Used in embedded systems to reduce memory usage while maintaining efficient state processing.

11. Optimization of DFA-based Pattern Matches

Used in search tools and firewalls to match patterns extremely fast in large data streams.

14. CONTENTS BEYOND SYLLABUS : UNIT - I

Tokenization in Modern Languages :

Tokenization (lexical analysis) is the process that converts a raw source file (a stream of characters) into a stream of tokens – atomic units with type and often attributes (lexeme text, line/column, literal value). Traditional lexer design assumes:

- Tokens are described by regular expressions,
- The lexer follows the maximal munch (longest-match) rule,
- The lexer is context-free (a DFA suffices).

Modern languages still use these basics, but many require extensions: Unicode handling, contextual keywords, significant whitespace, macro systems, nested/conditional lexing, string interpolation, and more.

General advanced tokenization concerns (applies to most modern languages)

1. Unicode & Identifiers

- Accepting non-ASCII identifier characters (e.g., Greek letters, emoji in some languages).
- Normalization (NFC vs NFD): two visually identical strings may be different sequences – many languages normalize to NFC for identifiers.

2. Maximal munch + Disambiguation

- Longest-match still applies, but sometimes context (e.g., / as division vs regex in JS) determines the token.

3. Contextual keywords

- Words that are identifiers in some contexts and keywords in others (e.g., async/await in some languages). Tokenizer often needs minimal parse-context.

4. Lexer states / modes

- Lexers switch modes for multi-line comments, string interpolation, template literals, raw strings, preprocessor conditions, or inside macros

5. Significant whitespace

- Languages like Python generate INDENT/DEDENT tokens based on changes to leading whitespace. This requires managing a stack of indentation levels, insertion of NEWLINE tokens, and dealing with line continuations.

14. CONTENTS BEYOND SYLLABUS : UNIT - I

6. String interpolation & multi-mode strings

- Strings with embedded expressions require the lexer to produce tokens for string content and then switch to expression lexing, then back.

7. Macro/preprocessor lexing

- Macro systems (C/C++ preprocessor, Rust macros) often require token streams that preserve more information (including macro tokens, token-pasting operators, and expanded locations).

8. Raw / alternate literal syntaxes

- Languages support raw strings (r"..."), multi-line raw strings, or delimited strings ("""...""") that change escape handling and may allow nested delimiters.

9. Comments + doc-comments

- Some doc-comments are structured (Markdown, XML) and need to be tokenized/preserved for tools (documentation generators).

10. Numeric literal varieties

- Binary/hex/octal forms, digit separators (e.g., 1_000_000), suffixes for types, and rational/complex literal syntax.

11. Performance & memory

- Incremental lexing (for IDEs), lazy token creation, memory-mapped input, and avoiding expensive Unicode operations unless needed.

12. Error recovery

- Tokenizers for IDEs must be tolerant – produce tokens even if the source is incomplete or ill-formed.

Example Python

1. Unicode & Identifiers

- Modern languages allow Unicode characters in identifiers.
- Python identifiers can include letters, digits, and underscore – but also Unicode letters like:

`π = 3.14`

`नाम = "Name"`

The tokenizer must:

- classify Unicode categories (letters)
- normalize text to a consistent Unicode form (NFC)

14. CONTENTS BEYOND SYLLABUS : UNIT - I

2. Maximal munch + Disambiguation

- Lexers always take the longest valid token (maximal munch).

>>> = >> =

Tokenized as: >>= # right-shift assignment Not: >> =

Python lexer chooses longest match

3. Contextual keywords

- A word may be a keyword only in some contexts.

Python added keyword `async`, `await`, but still allows:

`async = 10` # valid in older Python or special modes

The lexer must check:

Feature version

Context (type of statement)

4. Lexer states / modes

- Many languages switch lexing modes (strings, comments, interpolation).

Inside f-strings, the lexer switches:

string mode

expression mode inside {}

Example:

`f"Hello {name.upper()}"`

Lexer must:

- Tokenize string text
- Enter expression mode for `name.upper()`
- Return to string mode after }

5. Significant whitespace

Python uses indentation to define blocks.

if x:

`print("hi")`

Python tokenizer outputs:

INDENT

DEDENT

Based on changes in indentation level.

Lexer must maintain a stack:

[0] → [4] → back to [0]

6. String interpolation & multi-mode strings

Strings containing embedded expressions require switching lexing modes.

`f"{x+1} apples"`

The lexer must:

detect {

temporarily tokenize Python expressions

return to string-mode

If braces are doubled {{, it's literal and not interpolation

14. CONTENTS BEYOND SYLLABUS : UNIT - I

7. Macro/preprocessor lexing

- Python does not have a macro preprocessor, but a comparable feature is:
 - Docstrings (""" """) and decorators (@func) influence parsing behavior. Though not macros, they force the lexer to preserve:
 - String literal indentation
 - Decorator token sequences (@, identifier)

8. Raw / alternate literal syntaxes

Modern languages provide raw string forms.

```
r"C:\newfolder\test"
```

In raw strings, backslashes are not escapes.

Lexer must:

- Treat \n as literal characters

- Avoid interpreting escape sequences

Python also supports triple-quoted strings for multi-line content.

9. Comments + doc-comments

Comments may carry structured metadata.

Python docstrings:

```
def f():
```

```
    """This is a docstring."""
```

The tokenizer must:

- Treat this as a STRING token

- But tools like Sphinx expect it preserved for documentation

10. Numeric literal varieties

Modern languages allow multiple numeric formats.

```
10
```

```
0xFF
```

```
0b1011
```

```
1_000_000
```

```
3.14e-10
```

Lexer must interpret:

- digit separators _

- base prefixes (0x, 0b)

- exponent part

14. CONTENTS BEYOND SYLLABUS : UNIT - I

11. Performance & memory

Large files require efficient tokenization.

CPython uses:

- buffered reading

- caching of lookahead

- efficient line-to-token mapping for error messages

Tools like PyPy use just-in-time tokenization for performance.

12. Error recovery

Tokenizers must handle incomplete or malformed code gracefully.

Unclosed string:

```
"hello
```

Lexer emits:

```
ERRORTOKEN
```

but continues lexing so the editor/compiler can show further errors.

Summary

Below is a compact table summarizing each concern with its Python example:

Concern	Python Example
Unicode & Identifiers	$\pi = 3.14$
Maximal munch	<code>>>= token</code>
Contextual keywords	async keyword vs identifier
Lexer modes	f-string expression interpolation
Significant whitespace	INDENT / DEDENT
String interpolation	<code>f"{x}"</code>
Macro-like behavior	decorators, docstrings
Raw strings	<code>r"\n"</code>
Comments/docstrings	<code>"""doc"""</code>
Numeric literal types	<code>1_000</code> , <code>0b1010</code>
Performance concerns	buffered reading
Error recovery	unclosed string → ERRORTOKEN

15. ASSESSMENT SCHEDULE

- Tentative schedule for the Assessment During 2025-2026 even semester

S.NO	Name of the Assessment	Date	Portion
1	Unit Test 1	02.01.2026	UNIT 1
2	IAT 1	29.01.2026	UNIT 1 & 2
3	Unit Test 2	20.02.2026	UNIT 3
4	IAT 2	10.03.2026	UNIT 3 & 4
5	Model	08.04.2026	ALL 5 UNITS

16. PRESCRIBED TEXT BOOKS & REFERENCE BOOKS

•TEXT BOOKS :

- Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, Compilers: Principles, Techniques and Tools , Second Edition, Pearson Education, 2009.

REFERENCEBOOKS:

- Randy Allen, Ken Kennedy, Optimizing Compilers for Modern Architectures: A Dependence based Approach, Morgan Kaufmann Publishers, 2002.
- Steven S. Muchnick, Advanced Compiler Design and Implementation, Morgan Kaufmann Publishers -Elsevier Science, India, Indian Reprint 2003.
- Keith D Cooper and Linda Torczon, Engineering a Compiler, Morgan Kaufmann Publishers Elsevier Science, 2004.
- V. Raghavan, Principles of Compiler Design , Tata McGraw Hill Education Publishers,2010.
- Allen I. Holub, Compiler Design in C , Prentice-Hall Software Series, 1993.

17. MINI PROJECT SUGGESTION

1. Spam Keyword Detector

Description:

Design a simple spam detector using finite automata that recognizes spam words in an email.

Topics covered: DFA, pattern matching.

2. Chatbot Word Classifier

Description:

Create a simple chatbot pre-processor that tokenizes input sentences and identifies nouns, verbs, numbers, etc.

Topics covered: Token recognition, RE.

3. LEX-Based Calculator

Description:

Use LEX to build a scanner for a mini calculator that identifies numbers, operators, and evaluates expressions.

Topics covered: LEX, token specification, lexical analysis.

4. Email / URL Validator

Description:

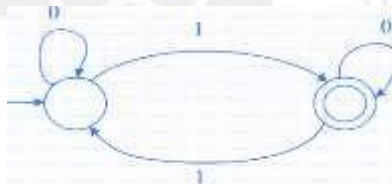
Translate a very small custom language into C statements, using lexical analysis + token processing.

Topics covered: Tokens, pattern matching, lexical design.

18. GATE QUESTIONS

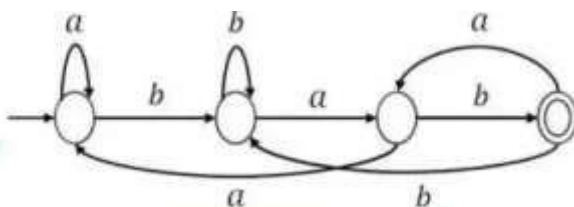
UNIT – I

1. Lex is an
 - a) Programming Language
 - b) Pattern Recognition Language**
 - c) Assembly Language
 - d) Macro
2. In a compiler, keywords of a language are recognized during
 - a) Parsing of the program
 - b) The code generation
 - c) The lexical analysis of the program**
 - d) Dataflow Analysis
3. A regular language L is accepted by a non-deterministic finite automaton (NFA) with n states. Which of the following statement(s) is/are FALSE?
 - a) L may have an accepting NFA with $< n$ states.
 - b) L may have an accepting DFA with $< n$ states.
 - c) There exists a DFA with $< 2n$ states that accepts L .
 - d) Every DFA that accepts L has $> 2^n$ states.**
4. Which one of the following regular expressions is equivalent to the language accepted by the DFA given below?



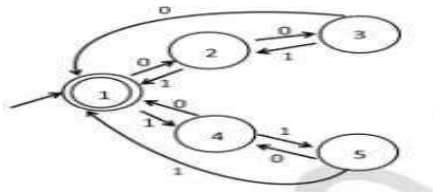
- a) $0^*1(0+10^*1)^*$ b) $1^*0(0+10^*1)^*$ c) $0^*1(1+10^*1)^*$ d) $0^*1(0+01^*1)^*$

5. Consider the following deterministic finite automaton (DFA) defined over the alphabet, $\Sigma = \{a, b\}$. Identify which of the following language(s) is/are accepted by the given DFA.



18. GATE QUESTIONS

1. The set of all strings containing an even number of b 's.
2. The set of all strings containing the pattern bab .
3. The set of all strings ending with the pattern bab .
4. The set of all strings not containing the pattern aba .
6. Consider the 5 -state DFA M accepting the language $L(M) \subset (0+1)^*$ shown below. For any string $w \in (0+1)^*$ let $n_0(w)$ be the number of 0's in w and $n_1(w)$ be the number of 1 's in w .



Which of the following statements is/are FALSE?

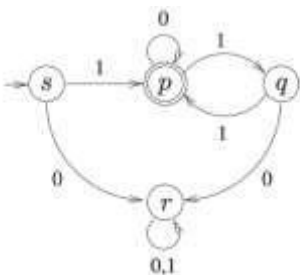
1. States 2 and 4 are distinguishable in M .
 2. States 3 and 4 are distinguishable in M
 3. States 2 and 5 are distinguishable in M
 4. Any string w with $n_0(w)=n_1(w)$ is in $L(M)$
7. The lexical analysis for a modern computer language such as Java needs the power of which one of the following machine models in a necessary and sufficient sense?

- a) Finite state automata b) Deterministic pushdown automata
c) Non-Deterministic pushdown automata d) Turing machine

8. Definition of a language L with alphabet $\{a\}$ is given as following $\{ a^{nk} \mid k > 0, \text{ and } n \text{ is a positive integer constant} \}$ What is the minimum number of states needed in a DFA to recognize L ?

- a) $k+1$ b) $n+1$ c) 2^{n+1} d) 2^{k+1}

9. Consider the Deterministic Finite-state Automaton (DFA) A is shown below. The runs on the alphabet $\{0,1\}$, and has the set of states $\{s,p,q,r\}$, with s being the start state and p being the only final state.



Which one of the following regular expressions correctly describes the language accepted by A ?

- a) $1(0^*11)^*$ b) $0(0+1)^*$
c) $1(0+11)^*$ d) $1(110)^*$

10. Is it possible to convert a NFA having 'n' states to some DFA having less than 'n' states?

- a) Yes b) No

Thank you



R.M.K.
GROUP OF
INSTITUTIONS

Disclaimer

:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the

respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail.

Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.